

Top 45 TypeScript Interview Questions & Answers (2026)

Frontend Developer — Fresher Edition

Basics

Ques 01: What is TypeScript and why do we use it over JavaScript?

Ans 01: TypeScript is a superset of JavaScript created by Microsoft. It adds types to JavaScript, so we can catch errors before running the code. We use it because it makes our code safer, easier to read, and easier to debug in large projects.

Example:

```
// JavaScript - no type checking
let age = 22;
age = "twenty-two"; // No error, but wrong

// TypeScript - type checking
let age: number = 22;
age = "twenty-two"; // ERROR - Type 'string' is not assignable to 'number'
```

Ques 02: What are the advantages of TypeScript?

Ans 02: TypeScript catches errors at compile time, before running the code. It gives better autocomplete and suggestions in the code editor. It also makes big projects easier to maintain because every variable and function has a clear type.

Example:

```
function greet(name: string) {
  return "Hello " + name;
}

greet(123); // ERROR caught immediately - editor shows red underline
greet("Rahul"); // Works fine
```

Ques 03: Is TypeScript compiled or interpreted?

Ans 03: TypeScript is compiled. Browsers cannot understand TypeScript directly, so a compiler called tsc converts TypeScript code into normal JavaScript code first. Then the browser runs that JavaScript.

Example:

```
// app.ts (TypeScript file)
let message: string = "Hello TypeScript";
console.log(message);

// Command to compile
// tsc app.ts
// This creates app.js (normal JavaScript) which browser can run
```

Ques 04: What is Type Inference in TypeScript?

Ans 04: Type inference means TypeScript automatically guesses the type of a variable based on its value, even if we don't write the type ourselves. We don't always need to declare types manually.

Example:

```
let name = "Rahul"; // TypeScript infers this as string automatically
let age = 22; // TypeScript infers this as number automatically

name = 22; // ERROR - even without writing ": string", TS knows it's a string
```

Ques 05: What are the basic data types in TypeScript?

Ans 05: TypeScript has all the basic JavaScript types plus its own extra types. The common basic types are string, number, boolean, array, and any.

Example:

```
let userName: string = "Pawan";
let age: number = 23;
let isLoggedIn: boolean = true;
let fruits: string[] = ["Apple", "Mango"];
let anything: any = "can be anything";
```

Ques 06: What is the any type and when should we avoid it?

Ans 06: any type means the variable can hold any kind of value, like plain JavaScript. We should avoid it because it removes all the type safety benefits of TypeScript. Using any too much defeats the purpose of using TypeScript.

Example:

```
let data: any = "Hello";
data = 42; // No error
data = true; // No error too
data = { name: "Rahul" }; // Still no error - this is risky

// Better - use specific type instead
let data2: string = "Hello"; // Now TS will catch wrong usage
```

Ques 07: What is the unknown type and how is it different from any?

Ans 07: unknown is similar to any because it can also hold any type of value. But unknown is safer — we cannot use an unknown value directly until we check its type first. any allows anything without any check.

Example:

```
let value: unknown = "Hello";

// value.toUpperCase(); // ERROR - must check type first with unknown

if (typeof value === "string") {
  console.log(value.toUpperCase()); // Works - after checking type
}
```

Ques 08: What is the void type used for?

Ans 08: void type is used for functions that do not return any value. It tells TypeScript that this function only performs an action but doesn't send back a result.

Example:

```
function logMessage(message: string): void {
  console.log(message);
  // no return statement
}

logMessage("Hello World"); // Just prints, returns nothing
```

Ques 09: What is never type in TypeScript?

Ans 09: never type is used for a function that never returns anything at all — like a function that always throws an error or runs forever in a loop. It is different from void because void returns undefined, but never doesn't return at all.

Example:

```
function throwError(message: string): never {
    throw new Error(message);
    // this function never completes normally
}

function infiniteLoop(): never {
    while (true) {
        console.log("Running forever");
    }
}
```

Ques 10: How do you define an array type in TypeScript?

Ans 10: We can define an array type by writing the data type followed by square brackets []. This means the array can only hold values of that specific type.

Example:

```
let numbers: number[] = [1, 2, 3, 4];
let names: string[] = ["Rahul", "Priya"];

numbers.push(5); // Works fine
numbers.push("six"); // ERROR - only numbers allowed

// Another way to write array type
let scores: Array<number> = [90, 85, 78];
```

Ques 11: What are Tuples in TypeScript?

Ans 11: Tuple is a special type of array where we fix the number of elements and their type in a specific order. Unlike normal array, each position has its own defined type.

Example:

```
// Tuple - fixed order: string then number
let user: [string, number] = ["Rahul", 22];

console.log(user[0]); // "Rahul"
console.log(user[1]); // 22

user = [22, "Rahul"]; // ERROR - wrong order
```

Ques 12: What are Enums in TypeScript?

Ans 12: Enum means a set of named constants. It is used when a variable can only have a few fixed values, like days of the week or status types. It makes code more readable than using plain numbers or strings.

Example:

```
enum Status {
    Pending,
    Approved,
    Rejected
}

let orderStatus: Status = Status.Approved;
console.log(orderStatus); // 1 (Approved is at index 1)

// String enum
enum Role {
    Admin = "ADMIN",
    User = "USER"
}
let myRole: Role = Role.Admin;
```

Interfaces & Types

Ques 13: What is an Interface in TypeScript?

Ans 13: Interface is used to define the shape of an object. It says the properties and types of the object, and the object follows this interface rule.

Example:

```
interface User {
    name: string;
    age: number;
}

let person: User = {
    name: "Pawan",
    age: 23
};

// person.age = "23"; // ERROR - age must be a number, not a string
```

Ques 14: What is the difference between type and interface?

Ans 14: Both are used to define the shape of data, but interface is mostly used for objects and we can add more properties to it easily. Type is more flexible – it can also define unions, primitives, and tuples, not just objects.

Example:

```
// interface - good for objects, can extend

interface User {
  name: string;
}

interface Admin extends User {
  role: string;
}

// type - more flexible, can do union too
type ID = string | number;

type Person = {
  name: string;
};
```

Ques 15: Can an interface extend another interface?

Ans 15: Yes, an interface can extend another interface using the extends keyword. This means the new interface gets all the properties of the old one, plus its own extra properties.

Example:

```
interface Animal {
  name: string;
}

interface Dog extends Animal {
  breed: string;
}

let myDog: Dog = {
  name: "Bruno",
  breed: "Labrador"
};
```

Ques 16: What are Optional Properties in an interface?

Ans 16: Optional properties are properties that may or may not be present in an object. We add a ? after the property name to make it optional. If we don't add ?, that property becomes required.

Example:

```
interface User {
  name: string;
  email?: string; // optional - can be skipped
}

let user1: User = { name: "Rahul" }; // Works - email skipped
let user2: User = { name: "Priya", email: "priya@gmail.com" }; // Also works
```

Ques 17: What are Readonly properties in TypeScript?

Ans 17: Readonly properties means the value can be set only one time. After that, we cannot change it. We write readonly before the property name.

Example:

```
interface User {
  readonly id: number;
  name: string;
}

let user: User = { id: 101, name: "Pawan" };

user.name = "Rahul"; // Works fine
// user.id = 102; // ERROR - cannot change readonly property
```

Ques 18: What is a Union Type?

Ans 18: Union type means a variable can hold more than one type. We use (|) pipe symbol to join the types together.

Example:

```
let id: string | number;

id = 101; // Works
id = "ABC101"; // Works too
// id = true; // ERROR

function printId(id: string | number) {
```

```
console.log("Your ID is: " + id);  
}
```

Ques 19: What is an Intersection Type?

Ans 19: Intersection type combines multiple types into one. The final type has all the properties from every type combined together. We use the & (ampersand) symbol.

Example:

```
type Person = {  
  name: string;  
};  
  
type Employee = {  
  salary: number;  
};  
  
type Staff = Person & Employee; // combines both  
  
let worker: Staff = {  
  name: "Pawan",  
  salary: 50000  
};
```

Ques 20: What are Type Aliases?

Ans 20: Type alias is a way to give a custom name to a type, so we can reuse it anywhere instead of writing the full type again and again. We use the type keyword to create one.

Example:

```
type ID = string | number;  
  
let userId: ID = 101;  
let productId: ID = "P-2023";  
  
type User = {  
  name: string;  
  age: number;  
};  
  
let person: User = { name: "Rahul", age: 22 };
```

Functions

Ques 21: How do you type function parameters and return values?

Ans 21: We write the type after the parameter name using a colon. We also write the return type after the function parentheses, again using a colon.

Example:

```
function add(a: number, b: number): number {  
    return a + b;  
}  
  
add(5, 3); // 8  
// add("5", 3); // ERROR - "5" is a string, not a number
```

Ques 22: What are optional parameters in a function?

Ans 22: Optional parameters are parameters that we don't have to pass every time we call the function. We add a ? after the parameter name to make it optional.

Example:

```
function greet(name: string, age?: number) {  
    if (age) {  
        console.log(name + " is " + age + " years old");  
    } else {  
        console.log("Hello " + name);  
    }  
}  
  
greet("Rahul"); // Works - age skipped  
greet("Priya", 25); // Also works
```

Ques 23: What are default parameters in TypeScript functions?

Ans 23: Default parameters are parameters that already have a value set. If we don't pass a value for that parameter, it uses the default value automatically.

Example:

```
function greet(name: string, city: string = "Delhi") {  
    console.log(name + " is from " + city);  
}
```

```
greet("Rahul"); // "Rahul is from Delhi" - default used
greet("Priya", "Mumbai"); // "Priya is from Mumbai" - default replaced
```

Ques 24: What is a function type/signature in TypeScript?

Ans 24: Function type tells us what type of parameters a function takes and what type of value it returns, without writing the actual function body. It is mostly used when we store a function inside a variable.

Example:

```
let add: (a: number, b: number) => number;

add = function (a, b) {
  return a + b;
};

console.log(add(4, 5)); // 9
```

Generics

Ques 25: What are Generics in TypeScript and why do we need them?

Ans 25: Generics let us write a function or component that can work with any data type, without losing type safety. We use them so we don't have to write the same code again for different types.

Example:

```
function getFirst<T>(arr: T[]): T {
  return arr[0];
}

console.log(getFirst<number>([1, 2, 3])); // 1
console.log(getFirst<string>(["Rahul", "Priya"])); // Rahul
```

Ques 26: How do you create a generic function?

Ans 26: We create a generic function using a type placeholder like <T>(type parameter) right after the function name. This T acts like a variable for the type, and when we call the function, TypeScript figures out the actual type.

Angle brackets T = <T>

Example:

```
function identity<T>(value: T): T {
    return value;
}
console.log(identity<string>("Hello"));
```

Ques 27: What are Generic Interfaces?

Ans 27: Generic Interface is an interface that uses a type placeholder like <T>, so it can work with different data types instead of just one fixed type.

Example:

```
interface Box<T> {
    value: T;
}

let numberBox: Box<number> = { value: 100 };
let stringBox: Box<string> = { value: "Hello" };
```

Classes & OOP

Ques 28: How do you define a class in TypeScript?

Ans 28: We define a class using the class keyword, just like in JavaScript, but we also add types to the properties and methods.

Example:

```
class Person {
    name: string;
    age: number;

    constructor(name: string, age: number) {
        this.name = name;
        this.age = age;
    }

    greet(): void {
        console.log("Hi, I am " + this.name);
    }
}
```

```
let p1 = new Person("Pawan", 23);
p1.greet(); // Hi, I am Pawan
```

Ques 29: What are access modifiers (public, private, protected)?

Ans 29: Access modifiers control where a class property or method can be used from. public means anyone can access it (this is default). private means it can only be used inside that same class. protected means it can be used inside the class and its child classes.

Example:

```
class User {
  public name: string;
  private password: string;
  protected age: number;

  constructor(name: string, password: string, age: number) {
    this.name = name;
    this.password = password;
    this.age = age;
  }
}

let user = new User("Rahul", "1234", 22);
console.log(user.name); // Works - public
// console.log(user.password); // ERROR - private, cannot access outside
```

Ques 30: What is the difference between an interface and an abstract class?

Ans 30: Interface only defines the shape of an object – it has no actual code inside. Abstract class can have both defined methods (with code) and abstract methods (without code) that child classes must complete.

Example:

```
// Interface - only shape, no code
interface Shape {
  area(): number;
}

// Abstract class - can have real code too
abstract class Animal {
  abstract makeSound(): void; // must be implemented by child

  eat(): void {
    console.log("Eating..."); // already has code
  }
}
```

```
}  
}
```

React + TypeScript

Ques 31: How do you type props in a React functional component?

Ans 31: We create an interface that lists all the props with their types, then we use that interface in the component's function parameter.

Example:

```
interface UserProps {  
  name: string;  
  age: number;  
}  
  
function User({ name, age }: UserProps) {  
  return <p>{name} is {age} years old</p>;  
}
```

Ques 32: How do you type useState in TypeScript?

Ans 32: We write the type inside angle brackets <> right after useState. This tells TypeScript what type of value this state will hold.

Example:

```
const [count, setCount] = useState<number>(0);  
const [name, setName] = useState<string>("");  
  
setCount(5); // Works  
// setCount("five"); // ERROR - must be a number
```

Ques 33: How do you type event handlers (like onClick, onChange) in React?

Ans 33: We use React's built-in event types, like React.MouseEvent for clicks and React.ChangeEvent for input changes. We write the type as the parameter type in the handler function.

Example:

```
function handleClick(e: React.MouseEvent<HTMLButtonElement>) {
  console.log("Button clicked");
}

function handleChange(e: React.ChangeEvent<HTMLInputElement>) {
  console.log(e.target.value);
}

<button onClick={handleClick}>Click Me</button>
<input onChange={handleChange} />
```

Ques 34: How do you type useRef in TypeScript?

Ans 34: We write the type of the element inside angle brackets <> after useRef, and set the initial value as null.

Example:

```
const inputRef = useRef<HTMLInputElement>(null);

function focusInput() {
  inputRef.current?.focus();
}

<input ref={inputRef} />
```

Ques 35: How do you type children props in a component?

Ans 35: We use the built-in type React.ReactNode for children, because children can be text, numbers, elements, or a mix of all these.

Example:

```
interface CardProps {
  children: React.ReactNode;
}

function Card({ children }: CardProps) {
  return <div className="card">{children}</div>;
}

<Card>
  <h1>Hello</h1>
  <p>This is a card</p>
</Card>
```

Ques 36: How do you type an API response in a React component?

Ans 36: We create an interface that matches the shape of the data coming from the API, then use that interface as the type for our state.

Example:

```
interface User {
  id: number;
  name: string;
  email: string;
}

const [users, setUsers] = useState<User[]>([]);

useEffect(() => {
  axios.get("https://api.example.com/users")
    .then((res) => setUsers(res.data));
}, []);
```

Ques 37: What is the use of interface for defining component Props?

Ans 37: Interface tells us exactly what props a component expects and what type each prop should be. This helps catch mistakes early, like passing wrong data or missing a required prop.

Example:

```
interface ButtonProps {
  label: string;
  onClick: () => void;
}

function Button({ label, onClick }: ButtonProps) {
  return <button onClick={onClick}>{label}</button>;
}

// <Button label="Submit" /> // ERROR - onClick is missing
```

Ques 38: How do you handle optional props in a React component?

Ans 38: We add a ? after the prop name in the interface to make it optional. Then the component works fine even if that prop is not passed.

Example:

```
interface UserProps {
  name: string;
  age?: number; // optional
```

```

}

function User({ name, age }: UserProps) {
  return <p>{name} {age ? `is ${age} years old` : ""}</p>;
}

<User name="Rahul" /> // Works fine without age

```

Misc / Practical

Ques 39: What is Type Assertion in TypeScript (as keyword)?

Ans 39: Type assertion means telling TypeScript "trust me, I know the type of this value" even if TypeScript is not sure. We use the as keyword to do this.

Example:

```

let value: unknown = "Hello World";

let strLength = (value as string).length; // telling TS it's a string

console.log(strLength); // 11

```

Ques 40: What is the difference between null, undefined, and never in TypeScript?

Ans 40: null means a variable is intentionally set to empty. undefined means a variable has no value assigned yet. never means a function never returns any value at all, like a function that always throws an error.

Example:

```

let a: null = null;
let b: undefined = undefined;

function throwError(): never {
  throw new Error("Something went wrong");
}

```

Ques 41: What is a tsconfig.json file used for?

Ans 41: tsconfig.json is a configuration file that tells the TypeScript compiler how to compile our TypeScript code into JavaScript. It has settings like which folder to compile, which JavaScript version to target, and whether to use strict mode.

Example:

```
{
  "compilerOptions": {
    "target": "ES6",
    "strict": true,
    "outDir": "./dist",
    "rootDir": "./src"
  }
}
```

Ques 42: What are Type Guards in TypeScript?

Ans 42: Type Guard is a way to check what type a value actually is, before using it. This helps TypeScript understand which type we are working with, so it allows only the correct operations.

Example:

```
function printValue(value: string | number) {
  if (typeof value === "string") {
    console.log(value.toUpperCase()); // safe - it's a string
  } else {
    console.log(value.toFixed(2)); // safe - it's a number
  }
}
```

Ques 43: What is Type Narrowing?

Ans 43: Type Narrowing means TypeScript reduces a wider type (like a union type) to a more specific type, based on a check we do in our code, like using typeof or if conditions.

Example:

```
function getLength(value: string | number) {
  if (typeof value === "string") {
    return value.length; // TS knows it's a string here
  }
  return value.toString().length; // TS knows it's a number here
}
```

Ques 44: How does TypeScript help catch errors at compile time vs runtime?

Ans 44: TypeScript checks our code for type mistakes before the code even runs, this is called compile time. JavaScript only finds errors when the code actually runs, this is called runtime. So TypeScript helps us fix bugs earlier.

Example:

```
function add(a: number, b: number) {
    return a + b;
}

add(5, "10"); // ERROR caught at compile time - before running

// In plain JavaScript, this same mistake would only show up
// when the code runs, and might give wrong output like "510"
```

Ques 45: What is the difference between compiling TypeScript with strict mode on vs off?

Ans 45: strict mode turns on all the strict type-checking rules in TypeScript, so it catches more mistakes like null errors and missing types. When strict mode is off, TypeScript is more relaxed and allows some unsafe code without warning.

Example:

```
// tsconfig.json
{
  "compilerOptions": {
    "strict": true // catches more errors, safer code
  }
}

// With strict mode ON
let name: string;
console.log(name); // ERROR - used before assigning value

// With strict mode OFF
let name: string;
console.log(name); // No error, but might give undefined at runtime
```